School portal — rules for Claude (and for every teammate)

Claude Code reads this file on every session in this folder. Everyone on the team follows the same rules, so the code stays as simple and consistent as it is now.

The project

- A school portal: five portals (student, parent, teacher, management, admin) + the public landing page.
- Vanilla PHP 8 / HTML / CSS / JS. No frameworks or libraries unless the lead approves.
- Each page's **look is finished**. Backend work keeps the page looking exactly as it does now.
- The code must stay simple enough to explain line by line in the viva and to change live in the code check.

Tasks

- Each person's tasks: `docs/team/handover/{name}.md`. Work only inside your own task unless the lead says so.
- Branch from `dev`, open the pull request into `dev`. The lead (repo owner) reviews and merges every pull request, and alone merges `dev` into `main`. Nothing reaches `dev` or `main` without that review.

How we work

- One task = one branch = one pull request (`docs/team/GIT.md`). Small commits, one change each.
- Before building, read: the page, its components, CSS and JS — and the backend under it (route, controller, model, config) just as closely. Then read the task in `docs/CHECKLIST.md` ("Backends left to connect") and your handover file. Use `/next-feature <task>` for the plan; build only after the lead approves the plan.
- "Don't build yet" / "discuss" = no file changes.
- Track work in `docs/CHECKLIST.md`: tick items when done and verified; add new findings there.
- Verify every change: `php -l` on PHP, `bash bin/check.sh` (0 errors), open the page in the browser (http://localhost:8080) and compare it with how it looked before. For anything that takes a form or a background request, also attack it with curl: no CSRF token, `field[]=x`, the wrong role, signed out. Reading code is not testing it.
- Report honestly: what works, what is still example data (mock), what was not checked.

Code rules

- Views in **per-role folders** (`app/Views/student/dashboard.php`…). Shared pieces are components in `app/Views/components/`; per-use differences come from a `$variant` (multi-variant components), not copied files.
- One component = its PHP + its own CSS file + its own JS file. `global.css` only holds tokens, resets and the app shell.
- Page rows (how components line up together) live in `public/assets/css/layout/{row}-grid.css` with `l-…-grid` classes, e.g. `l-chart-calendar-grid`; a page lists its rows in `$grids`. Only arranging code goes there (grid, columns, gaps, screen sizes, space between rows) — never a component's own look.
- One name per colour (`global.css` tokens). No duplicate rules, no `!important`, no inline `style=""`, no dead code. Dead code includes: a file no page loads, a parameter nobody passes, a setting / colour token / route nobody uses. One rule in one place: role lists and "who may do what" live in `config/permissions.php` (or the feature's config).

- Models are the only place with SQL (prepared statements). Controllers decide. Views only display (`e()` on every value). SQL built from array keys only from a fixed list in the code. Several changes that belong together → `transaction()`.
- Form values only through `postText()` / `postPassword()` / `postList()`, never `$_POST` directly.
- CSRF is checked once, by `core/Router.php`, for every POST — never by hand. Logout is POST. A background (JSON) request gets JSON back on every path (`refuse()`); a crash shows `errors/error.php`, never code or file paths.
- Never reveal which accounts exist: an unknown ID and a real one get the same message. Anything that sends an email or a code has a wait between sends. Secrets (passwords, keys) live in `.env` only, never in code or in git.
- White-label: another school must work by changing `config/white_label/` only. Name, logo, contacts → `school()`; Grade / Class / Term / Student… → `word()` / `plural()`; modules → `feature()`; dates → `country('date_format')` (PHP) and `<html lang>` (JS); "today" = PHP's date. Never type the school's name, domain, phone code or words — not even in comments, JS names or storage keys. Every setting a feature needs is read from there.
- A school rule with a number in it (a pass mark, an age limit, a maximum) goes in `config/white_label/rules.php` (read with `rule('…')`) or the feature's config — never typed into a controller.
- Names match the screen: the words the page shows are the words in tables, models, methods and config keys (the page says "Audit Log" → the table is `audit_log`). Database changes are **updates**: `database/updates/NNN_name.sql`, run with `php database/update.php` — never edit an update that is already in `dev`.
- Formatting: PHP and JS `{` on the next line; CSS `selector {` on the same line, one property per line.

Explaining

Short, line by line, no long paragraphs. Show which file calls which and what is passed. Give complete code. The person asking must be able to explain every line themselves: if they can't, explain it again (`/explain`).

Running

- `README.md`: Docker MySQL on port 3308 (root / root, database `l_ecole`), phpMyAdmin http://localhost:8086, the site on http://localhost:8080. Demo accounts' password: `Password@123`.